Project Blueprint: SwiftStock Pro

Stack: Next.js 14 (App Router), Tailwind CSS, Framer Motion, Flask, PostgreSQL
Line Spacing: 1.25

1. Core Architectural Requirements

To move beyond "average," the application must implement professional patterns for data fetching, security, and state management.

Feature Tier	Implementation Detail	Learning Objective
Data Fetching	Next.js Server Components with React Query for client-side caching.	Caching & Hydration
Auth Strategy	NextAuth.js (Frontend) + JWT / Flask-JWT-Extended (Backend).	Secure Session Mgmt
Database	PostgreSQL with SQLAlchemy ORM and Alembic migrations.	Relational Modeling

2. Detailed Feature Roadmap

Phase 1: The Intelligent Dashboard

- **Real-time Analytics:** Cards showing Total Inventory Value, Low Stock Alerts (< 10 units), and Top Categories.
- **Data Visualization:** Use recharts to show stock movement over the last 30 days.
- **UI/UX:** Implement a "Skeleton Loader" while data is being fetched to prevent layout shift.

Phase 2: Advanced Inventory CRUD

- **Dynamic Data Table:** Server-side pagination, sorting, and multi-field filtering (Search by name, category, or SKU).
- **Bulk Actions:** Ability to select multiple items to "Batch Update" prices or "Export to CSV."
- **Image Management:** Drag-and-drop product image uploads with Flask handling file validation and storage.

Phase 3: The "Pro" Edge (Non-Average Features)

- **Optimistic UI:** When a user updates a stock count, the UI reflects it immediately while the Flask request happens in the background.
- **Activity Logs:** A backend table that records every change (Who changed what and when) to create an audit trail.
- **Dark Mode & Framer Motion:** Smooth layout transitions when navigating between views and a professional dark/light toggle.

3. Backend API Specification (Flask)

Define strict RESTful endpoints with proper HTTP status codes:

```
GET    /api/products           # Fetch all products (supports ?page=1&limit=10)
POST   /api/products           # Create new item (Validated by Marshmallow)
PATCH  /api/products/<id>      # Partial update (e.g., just stock count)
DELETE /api/products/<id>      # Soft delete (sets is_active=False)
GET    /api/analytics/summary # Aggregated stats for dashboard
```

4. Frontend Component Strategy (Next.js)

- **Reusable UI Components:** Build a custom <Button />, <Input />, and <Modal /> library using Tailwind.
- **Form Logic:** Use react-hook-form with zod for schema validation (e.g., Price cannot be negative).
- **Error Handling:** Implement Global Error Boundaries to catch API failures gracefully.

5. Success Metrics

The project is complete when:

1. The app passes a Lighthouse performance score of 90+.
2. A user can successfully log in, add a product with an image, and see it reflected on the dashboard stats instantly.
3. API requests are protected and return a 401 Unauthorized if no token is present.